

LARA: Training Inputs, Data Generation, and Losses

Alessandro Berti

Wil M. P. van der Aalst

A Training Data Generation

This standalone appendix specifies the concrete network inputs and training targets, expands the data generation procedure, and gives the complete loss used by the reference run. The data generator creates a supervised alignment corpus in which every row contains a Petri net, one trace, and the exact optimal alignment for that trace on that net. The relevant code path is `scripts/init_data.py`, which calls the behavior-family generator in `lara_align/families.py`, the PM4Py exact aligner in `lara_align/exact.py`, and the replay verifier in `lara_align/verify.py`. The checked-in reference corpus uses dataset format version 2 and the defaults represented in `configs/behavior_families.json`.

A.1 What One Training Row Contains

One generated row is an `AlignmentSample`. It contains:

- a unique sample id and its split;
- a Petri net SN with its initial and final markings;
- one observed trace σ ;
- an optimal alignment γ^* computed by the exact backend;
- the optimal cost $\delta = c(\gamma^*)$ under the standard move costs;
- metadata describing how the row was generated.

The standard costs are the same costs used throughout the paper: synchronous moves cost zero, visible log moves cost one, visible model moves cost one, and silent model moves cost zero. The exact alignment is transition-aware. Thus a synchronous move stores the concrete transition name, not only the visible activity label. This matters when two transitions have the same label.

The network receives only numeric features of the net, its markings and labels, and the observed trace. Sample IDs, family metadata, the clean trace, the exact alignment, and its cost are not input features. The neural training targets are derived from this exact alignment. For each trace event, the sync target is the transition fired by the exact alignment when that event is synchronized. Events skipped by the optimum become log-move targets. Transitions fired without consuming an event become model-move targets. The exact cost is also stored as the target for the learned cost estimate.

A.2 A Concrete Stored Training Input

Consider `train-000048` in `data/lara_synthetic/train.pkl` (seed 13, dataset version 2). It belongs to the duplicate-prefix representation of family `train-6fda2acef53502aa`. The generator selected the clean trace $\langle A, B, D, E \rangle$, swapped the last two events, and substituted C for B , yielding the observed input $\sigma = \langle A, C, E, D \rangle$. The two representations of this family both admit exactly $ABDE$ and $ACDE$; the same observation is separately aligned on each representation.

For readability, write A_L, A_R, B, C, D, E for transition identities, with $\lambda(A_L) = \lambda(A_R) = A$ and the other labels equal to the displayed letters. The complete input net has six places and six transitions. Its paths are

$$p_0 \xrightarrow{A_L} p_L \xrightarrow{B} p_f \xrightarrow{D} p_D \xrightarrow{E} p_E, \quad p_0 \xrightarrow{A_R} p_R \xrightarrow{C} p_f.$$

All arcs have weight one. The initial marking is $[p_0]$ and the final marking is $[p_E]$; p_f is only the end of the core, before the shared suffix. Table 1 fixes the actual sorted tensor order and connects the readable notation to the stored identifiers. Names fix indices but are not embedded as semantic features.

Table 1: Stored identifier order for `train-000048`. Node indices put places first; transition target indices are zero-based within the transition list.

node	place	stored name	target index	transition	stored name
0	p_D	context_sink_000	0	D	context_000_D
1	p_E	context_sink_001	1	E	context_001_E
2	p_0	p0	2	A_L	t_A_left
3	p_f	pf	3	A_R	t_A_right
4	p_L	pl	4	B	t_B
5	p_R	pr	5	C	t_C

Input tensors. `pm4py_to_features` in `lara_align/features.py` produces the input tuple $X = (X_P, X_T, E, r, z_T, z_\sigma, C)$. A place row is $(m_0, m_f, d^-, d^+, d^- + d^+)$; a transition row is $(1_\tau, d^-, d^+, c_M, c_S, s)$, where s records a shared input/output place. For this row,

$$X_P = \begin{bmatrix} 0 & 0 & 1 & 1 & 2 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 2 & 2 \\ 0 & 0 & 2 & 1 & 3 \\ 0 & 0 & 1 & 1 & 2 \\ 0 & 0 & 1 & 1 & 2 \end{bmatrix}, \quad X_T = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}.$$

The transition graph nodes are $6, \dots, 11$ in target-index order. The following table determines every column of E and every edge type r . For each row (p, t, q) , append the four directed edges $(p, t), (t, p), (t, q), (q, t)$, with types $(0, 2, 1, 3)$ respectively. Types 0, 1 follow consumption and pro-

duction arcs; types 2, 3 reverse them. Thus E is 2×24 ; r repeats $(0, 2, 1, 3)$ six times.

transition	p	t	q
D	3	6	0
E	0	7	1
A_L	2	8	4
A_R	2	9	5
B	4	10	3
C	5	11	3

For visible labels, the hash is $h(a) = 1 + (\text{int}(\text{BLAKE2b}_8(\text{UTF8}(a))) \bmod 8191)$, using an unsigned big-endian integer; $h(\tau) = 0$. The integer input vectors before augmentation are

$$z_T = (2587, 4475, 8075, 8075, 6408, 5878), \quad z_\sigma = (8075, 5878, 4475, 2587).$$

The shared embedding matrix has 8,192 learned rows, each of dimension 96. An ID selects one row; it is neither an activity count nor a numeric measure of similarity. Event positions $(0, 1, 2, 3)$ index a separate learned position embedding. Exact equality of the original visible labels gives

$$C = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

For example, the first event matches both A transitions; the third event matches E even though E is not enabled at the initial marking. Compatibility is a label relation, not an enabledness relation. All seven input tensors are now specified; no alignment or intermediate marking is supplied to the scorer.

Exact witness and target tensors. The stored optimal witness is

$$\gamma^* = \langle (A, A_R), (C, C), (E, \gg), (D, D), (\gg, E) \rangle, \quad \delta = 2.$$

It skips the observed E , synchronizes the final observed D , and completes with a model move E . Both the log move and visible model move cost one. An alternative optimum inserts model move D before synchronizing the observed E and skips the last observed D . Training uses the stored witness, not all optimal witnesses simultaneously.

With the transition indices in Table 1, `targets_from_alignment` in `lara_align/training.py` returns

$$y = (3, 5, -1, 0), \quad q = (0, 0, 1, 0), \quad b = (0, 1, 0, 0, 0, 0), \quad \delta = 2.$$

Here y is an integer tensor of length four, q a binary event target of length four, b a binary transition target of length six, and δ a scalar. The value -1 excludes that event from synchronous cross-entropy. A positive b_j means that transition j occurs at least once as a model move, including silent moves when present; it gives neither the position nor the number of such occurrences. These are supervision tensors, separate from X . The row contains C but not B , so transition-identity supervision is active.

Training-time relabeling. Each training visit independently remaps label IDs with probability 0.5. The implementation draws distinct replacement IDs from $1, \dots, 8191$ for the distinct nonzero IDs appearing in the net and trace, and applies the same mapping to both vectors. For example,

the assignment $8075 \mapsto 101$, $6408 \mapsto 205$, $5878 \mapsto 309$, $2587 \mapsto 413$, $4475 \mapsto 517$ would give $z'_T = (413, 517, 101, 101, 205, 309)$ and $z'_\sigma = (101, 309, 517, 413)$. This is an illustrative augmentation, not the recorded random draw. The graph, C , and all targets are unchanged; ID zero remains reserved for silent transitions. Existing hash collisions remain collisions. This trains reuse across activity vocabularies without claiming exact invariance to every label renaming. Validation and test use unmodified hash IDs.

A.3 Behavior Families

The generator does not create isolated trace and net pairs directly. It first creates a *behavior family*. Formally, a family fixes two labeled system nets SN_1, SN_2 whose complete visible languages agree, $L(SN_1) = L(SN_2) = L$, and two observations σ_1, σ_2 obtained by corrupting traces from L . An observation need not itself belong to L . This is the unit that is assigned to train, validation, or test before any row is written. Assigning whole families to a split keeps the four rows of a generated family together. It does not make motif classes or all visible languages disjoint across splits: independently generated families can share a language.

With the default configuration, each family has two Petri net representations and two observed traces. Therefore each family expands to four rows:

$$2 \text{ representations} \times 2 \text{ traces} = 4 \text{ rows.}$$

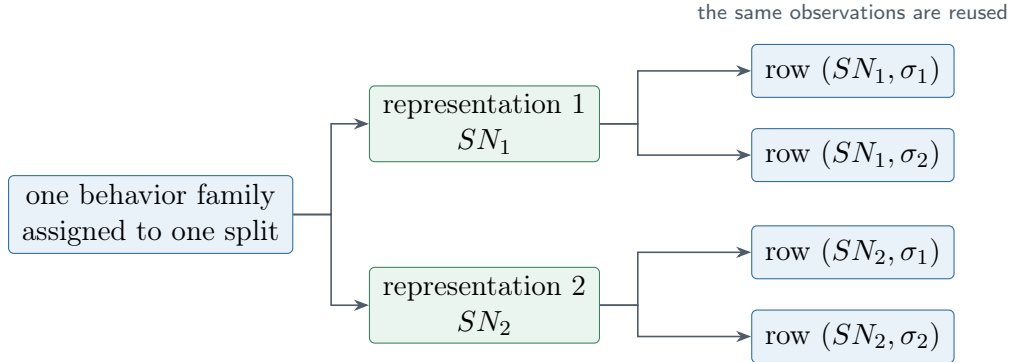


Figure 1: Expansion of one behavior family into four supervised rows. The split is chosen at family level, and both representations receive the same two observed traces.

The reference corpus uses four family motifs with equal weight. Table 2 summarizes them.

Before generation, the requested family count is converted into a deterministic motif quota. The quota reserves the configured per-motif minimum, distributes the remaining families according to the motif weights, and shuffles the plan with a split-specific seed. If an attempted family is rejected, the next family index is tried for the same planned motif. Consequently, rejection does not silently change the requested class balance. The family id, rather than an individual row id, is the split key.

Ordinary Tree. The ordinary-tree motif samples a random block-structured process tree. The default tree has between 4 and 12 visible activity occurrences and depth at most 6. Internal nodes are sampled from sequence, exclusive choice, and parallel composition with weights 0.4, 0.3, and 0.3. The visible alphabet size is about 70% of the number of visible occurrences, so duplicate labels can appear naturally. The first representation is the compiled block-structured Petri net. The second

Table 2: Behavior-family motifs used in the reference corpus. Each family contains two Petri net representations of the same visible behavior.

motif	representation 1	representation 2
ordinary tree	canonical block net	isomorphic renaming
duplicate versus silent	duplicate-prefix net	silent-routing net
concurrent versus interleaved	parallel net	explicit interleaving net
M-pattern	canonical block net	non-free-choice M-pattern net

representation is an isomorphic renaming of the same net: places and transition names are changed, but labels and behavior are preserved.

Figure 2 uses a small sampled tree to illustrate the transformation. In the Petri net figures, circles are places, blue rectangles are visible transitions, dark rectangles are silent transitions, and a bullet marks the initial token.

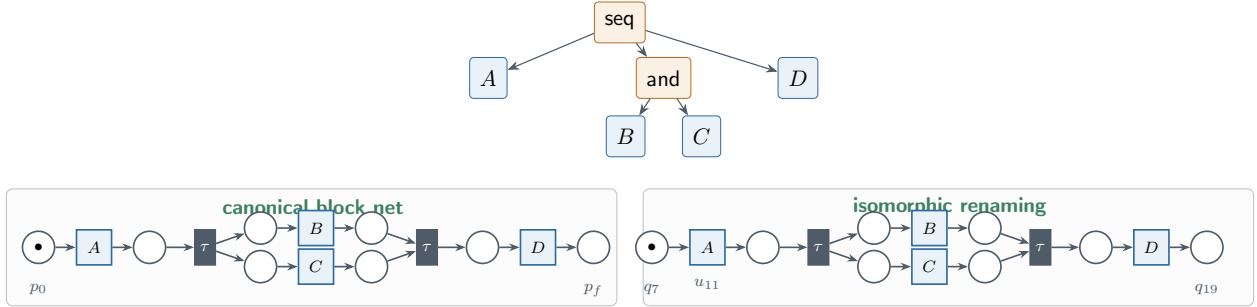


Figure 2: Illustrative ordinary-tree family for $\text{seq}(A, \text{and}(B, C), D)$. The second net preserves all arcs and visible labels while changing place and transition identities. Both nets have the complete visible language $\{\langle A, B, C, D \rangle, \langle A, C, B, D \rangle\}$.

Duplicate Versus Silent. This motif has two ways to represent the visible behavior “ A followed by either B or C ”. In the duplicate-prefix representation, there are two different visible transitions labeled A : one leads to the B branch and the other to the C branch. In the silent-routing representation, there is one visible A transition followed by a silent choice between the two branches. This motif directly tests whether the model learns transition identity rather than only label identity.

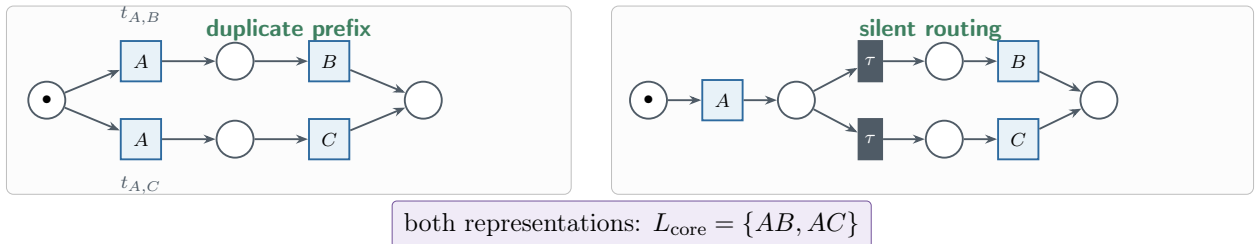


Figure 3: Two representations of the duplicate-versus-silent motif. The left net contains two concrete transitions with label A ; the right net contains one A transition and a silent choice.

Concurrent Versus Interleaved. This motif represents the visible language $\langle A, B \rangle$ and $\langle B, A \rangle$ in two different ways. One representation uses real parallelism: an invisible split produces two concurrent branches, and an invisible join closes them. The other representation encodes the two possible orders explicitly as alternatives. The two nets have the same visible complete traces but different causal structure.

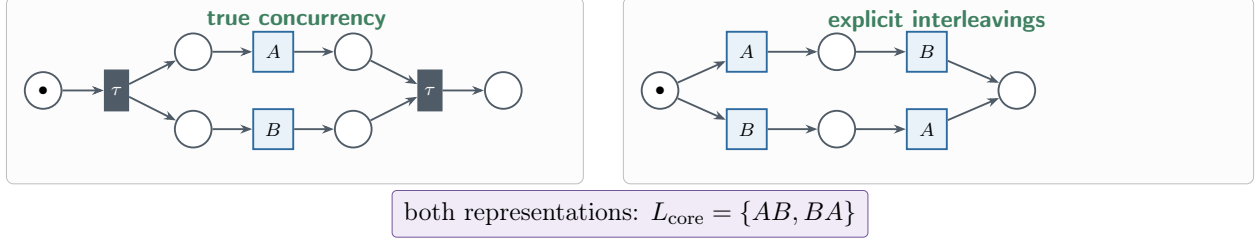


Figure 4: Two representations of the concurrent-versus-interleaved motif. Independent enabled transitions produce both orders on the left; an exclusive choice between two fixed paths produces the same visible language on the right.

M-pattern. This motif compares a block-structured net with a non-free-choice M-pattern. The visible behavior is an exclusive choice between B and the parallel execution of A and C . The non-free-choice representation shares input places in a way that violates free choice. This gives the model examples whose behavior is simple but whose Petri net structure is not block structured.

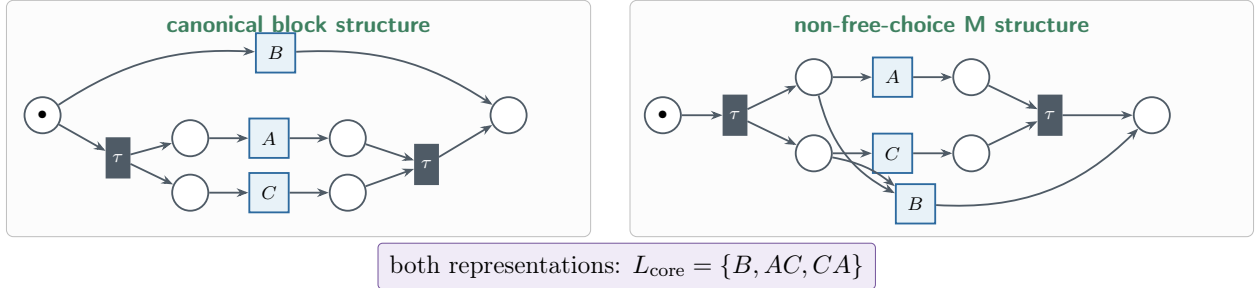


Figure 5: Two representations of the M-pattern motif. In the right net, A and B share one input place while B and C share another, so the choice is not free. The complete visible languages still agree.

For each of the three controlled motifs above, the default generator appends two visible activities in sequence, labeled D and E , to both representations. These labels are fresh relative to that core. If its language is L_{core} , the full language becomes $\{wDE \mid w \in L_{\text{core}}\}$. For example, $\{AB, AC\}$ becomes $\{ABDE, ACDE\}$, and $\{B, AC, CA\}$ becomes $\{BDE, ACDE, CADE\}$. Every clean trace gains two events; language equality between the paired nets is preserved. The suffix is attached to the core exit, and its end becomes the new final marking.

A.4 Checking That Representations Match

After the representations are built, the generator enumerates their visible complete-trace languages. The default limits for this check are 5,000 search states, 10,000 traces, and visible length 20; a separate firing bound prevents unbounded silent exploration. The certificate is **exact** only when the

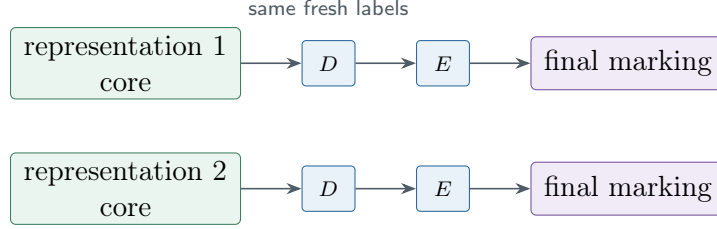


Figure 6: Shared visible suffix appended to both representations of each controlled motif. The suffix changes the full traces but not the comparison between representation structures.

enumeration finishes without hitting a bound, and is otherwise **bounded**. A family is rejected if the enumerated languages disagree. A family is also rejected if the reference language is empty, the final marking is unreachable, or any transition is dead. By default, bounded certificates are additionally rejected for training; presets intended for bounded loops can relax that requirement.

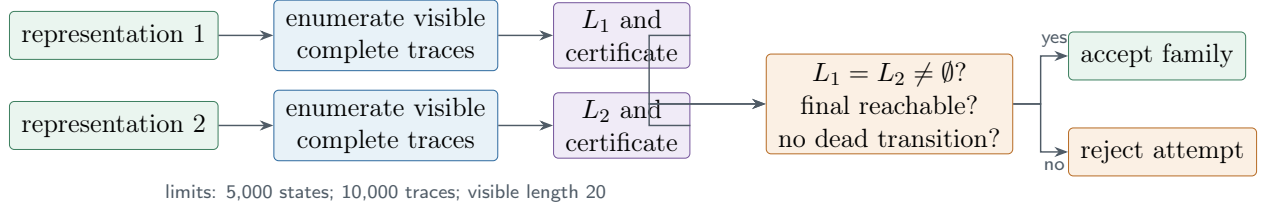


Figure 7: Family-level representation check. Enumeration records whether each language result is exact or bounded; default training additionally requires an exact certificate. A rejected attempt is replaced without changing its planned motif.

The reference corpus is small enough that all accepted train, validation, and test rows have an exact visible-language certificate. In other words, the language comparison completed within the limits for every accepted row.

A.5 Clean Trace Pool

For each accepted family, the generator sorts the enumerated visible language and keeps at most 16 clean traces. If the language has more than 16 traces, a deterministic sample of 16 indices is selected with the family model RNG and then restored to language order. These clean traces are not training rows yet. They are the fitting traces from which observed traces are produced.

The default configuration creates two observed traces per family. The first two clean traces in the selected pool are used, wrapping around if the pool is smaller than two. Trace generation is deterministic given the global seed, the split name, and the family index.

A.6 Trace Noise

Noise is added once at the visible-label level. The same noisy trace is then paired with every representation in the family. This is important: if a family has a duplicate-prefix net and a silent-routing net, both nets see the same observed trace. The exact aligner then decides how that same observation should be explained on each representation.

With the default noise settings, 20% of observed traces are kept clean. The remaining traces receive one, two, or three edits with probabilities 0.5, 0.3, and 0.2. The edit operations are:

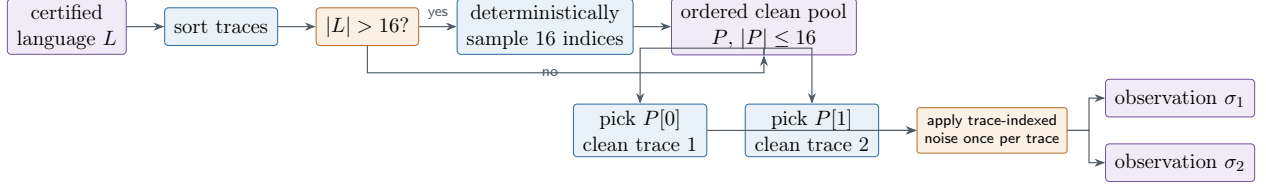


Figure 8: Construction of the two observations for an accepted family. A small pool is selected from the certified language, then each selected clean trace is corrupted once. The resulting σ_1 and σ_2 are reused for both representations.

- delete one event;
- insert an event whose label is already in the family alphabet;
- substitute one event label by another label from the family alphabet;
- swap two neighboring events;
- repeat one event;
- remove a prefix;
- remove a suffix;
- insert the special outside label `__OOD__`.

The first five operations have weight 0.15 each, prefix and suffix truncation have weight 0.10 each, and outside insertion has weight 0.05. Generated traces are capped at length 128. An operation is drawn independently at every edit. If the selected operation is inapplicable to the current trace (for example, a deletion from an empty trace or a swap in a one-event trace), the implementation falls back to outside insertion, so every requested edit still produces a recorded operation. Consequently, the realized outside-insertion frequency can be higher than its nominal weight on very short traces.

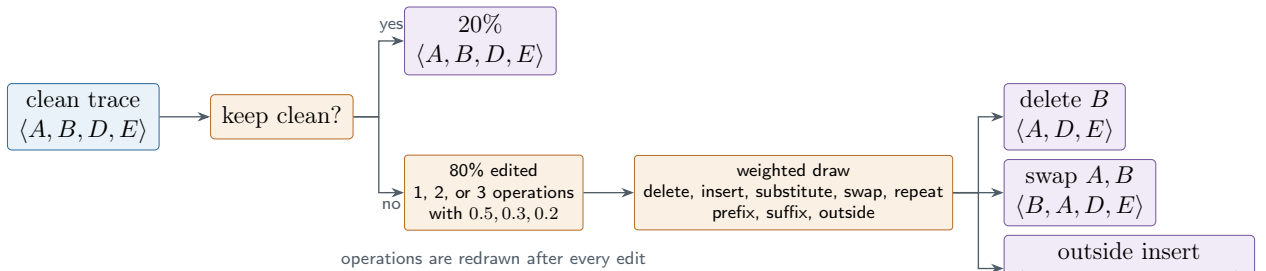


Figure 9: Default trace-corruption mechanism with three illustrative outcomes. The examples show individual edits; multi-edit observations repeatedly apply the same weighted operation draw.

A.7 Exact Labeling

The initializer expands every accepted family into trace and net pairs and sends each pair to the PM4Py state-equation A* aligner. The reference generation uses no per-trace timeout. The aligner is run with transition-aware output enabled, so the returned alignment identifies the actual Petri

net transition used by each synchronous or model move. Exact results are cached by net signature, observed label sequence, and cost-model version within each split.

Every exact result is replay-checked by LARA’s verifier. Replay checking starts from the initial marking, fires the model side of the alignment move by move, checks that the log projection equals the observed trace, and checks that the final marking is reached. If replay fails, the row is rejected. If PM4Py does not return an optimal alignment, the whole family is rejected and another family index is tried. Rows are buffered until every trace and representation pair in the family passes, so no partial family is written under the default strict coverage mode.

For families whose representation equivalence was certified exactly, the initializer also checks cost consistency: the same observed trace must have the same optimal cost on all equivalent representations. If equivalent representations disagree in cost, the family is rejected. This prevents the model from receiving contradictory targets for what is meant to be the same visible behavior.

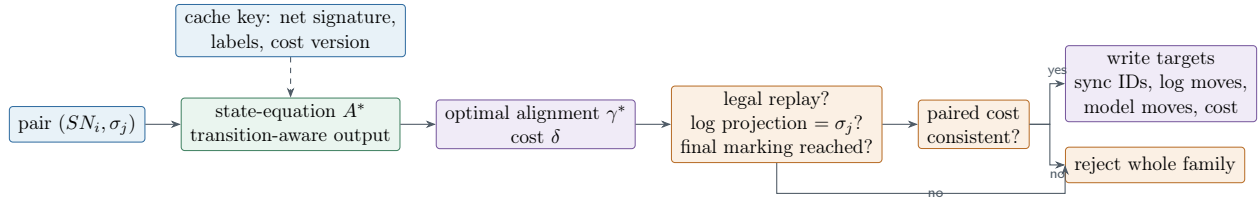


Figure 10: Exact-labeling and verification pipeline. Rows are committed only after every pair in the family passes replay; exactly certified equivalent representations must also agree on optimal cost.

A.8 Masked Transition-Identity Targets

Most synchronous targets can be used directly. One corner case occurs in the duplicate-prefix representation. Its A transition is considered identifiable exactly when the observed labels contain B or C , but not both. Noise can remove both distinguishing labels or introduce both, in which case a particular optimal A witness is not an identifiable target. The implementation then masks all synchronous transition-identity targets for that row from the sync loss. The row is still kept: log-move, model-move, and optimal-cost targets remain active, and the exact witness remains replay-verified. The metadata records this as `transition_identity_loss_masked`.

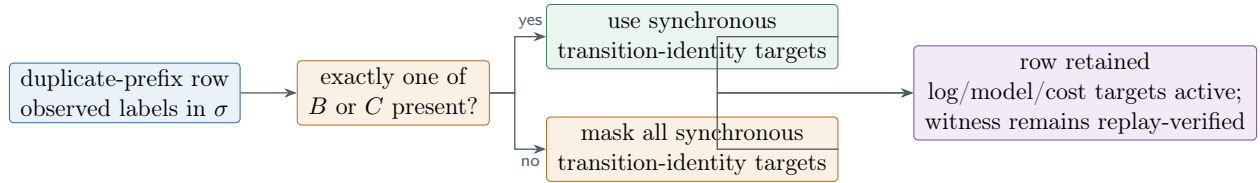


Figure 11: Masking rule for ambiguous duplicate-prefix observations. Masking removes only transition-identity supervision, not the row or its other exact targets.

A.9 Reference Split Sizes

The default command

```
python scripts/init_data.py -overwrite
```

creates the reference corpus in `data/lara_synthetic`. It writes the split files `train.pkl`, `val.pkl`, and `test.pkl`, plus `metadata.pkl` and `metadata.json`.

Table 3: Reference corpus generated with seed 13. Each family expands to four rows because it has two representations and two observed traces.

split	families	rows	families per motif
train	512	2,048	128
validation	128	512	32
test	128	512	32

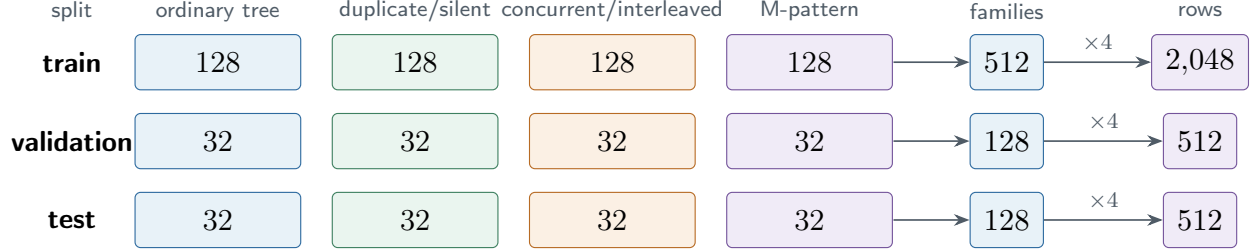


Figure 12: Balanced family quotas and row expansion in the reference corpus. Every family remains wholly inside its assigned split, and every accepted family contributes four rows.

Strict class coverage is used by default. This means the requested split size must contain complete families, and every active motif must meet its required minimum. In the reference corpus all four motifs are exactly balanced in every split. The manifest records one generation detail that does not affect the accepted corpus: 25 attempted families raised a generation-time `ValueError` and were replaced by later accepted families of the same planned motifs. No exact-alignment, replay, or paired-cost rejection was recorded for this run.

B Neural Scoring and the Complete Training Loss

B.1 From Input Tensors to Fixed Move Scores

With trained parameters θ , the encoder maps $X(SN, \lambda, \sigma)$ to event vectors v_i and transition vectors u_j , each of dimension $d = 96$. Place features receive a linear projection and a place-type embedding. Transition features receive a linear projection, a transition-type embedding, and the embedding indexed by z_T . Two graph blocks combine global self-attention and typed arc messages. For node v , the message term is

$$\mu_v = \frac{\sum_{(u,v,r) \in E} W_r h_u}{\max(1, |\{(u,v,r) \in E\}|)}.$$

Each block adds attention and μ_v through a residual connection and layer normalization, followed by a residual feed-forward block. A one-layer trace transformer processes label embeddings plus position embeddings with no causal mask. Two cross-attention modules let events attend to transitions and transitions attend to events. Hence both outputs depend on the whole trace, including future events, and on the net’s topology, labels, initial/final markings, and transition costs.

The heads in `lara_align/model.py` compute

$$\begin{aligned}\text{sync}_\theta(i, t_j \mid SN, \lambda, \sigma) &= S_{ij} = \begin{cases} (Wv_i)^\top u_j / \sqrt{d}, & C_{ij} = 1, \\ -10^9, & C_{ij} = 0, \end{cases} \\ \text{model}_\theta(t_j \mid SN, \lambda, \sigma) &= M_j = w_M^\top u_j + b_M, \\ \ell_i &= w_L^\top v_i + b_L.\end{aligned}$$

Dropout is applied during training and disabled at inference. At inference one forward pass gives $S \in \mathbb{R}^{n \times |T|}$, $M \in \mathbb{R}^{|T|}$, and $\ell \in \mathbb{R}^n$. The current decoder uses only S, M , reusing them at every visited marking. Its enabledness checks and bounded searches depend on the current marking; the neural scores do not. The model-move vector is conditioned on the trace but is not indexed by event position. The main paper’s worked decoding figure lists all move scores for a complete example, including the unused log scores.

B.2 Move Imitation

Use the target conventions of Section A.2. Let $I = \{i : y_i \geq 0\}$ and $J_i = \{j : C_{ij} = 1\}$. In the following equations, y_i identifies its corresponding transition column, accounting for the stored zero-based indices. For $i \in I$, define

$$p_{ij} = \frac{\exp S_{ij}}{\sum_{k \in J_i} \exp S_{ik}}, \quad j \in J_i.$$

The implementation uses a finite -10^9 sentinel outside J_i ; its softmax contribution underflows to zero. With smoothing $\epsilon_s = 0.05$,

$$\mathcal{L}_{\text{sync}} = -\frac{1}{|I|} \sum_{i \in I} \left[(1 - \epsilon_s) \log p_{i, y_i} + \frac{\epsilon_s}{|J_i|} \sum_{j \in J_i} \log p_{ij} \right].$$

Set this term to zero if I is empty. For the concrete row, $I = \{1, 2, 4\}$; only the first event offers two compatible transitions. Its smoothed target masses are 0.025 on A_L and 0.975 on A_R . Events with a unique compatible transition contribute zero synchronous cross-entropy.

For a raw logit z and binary target u , write $\text{BCE}(z, u) = \log(1 + e^z) - uz$, evaluated numerically in its stable form. With binary smoothing $\epsilon_b = 0.03$, set $\tilde{u} = (1 - 2\epsilon_b)u + \epsilon_b$. The other move losses are

$$\mathcal{L}_{\log} = \frac{1}{n} \sum_{i=1}^n \text{BCE}(\ell_i, 0.94q_i + 0.03), \quad \mathcal{L}_{\text{model}} = \frac{1}{|T|} \sum_{j=1}^{|T|} \text{BCE}(M_j, 0.94b_j + 0.03).$$

An empty output contributes zero. For the concrete row, smoothing gives

$$\tilde{q} = (0.03, 0.03, 0.97, 0.03), \quad \tilde{b} = (0.03, 0.97, 0.03, 0.03, 0.03, 0.03).$$

Log loss has the same weight as the other move losses, even though the decoder does not consume its scores.

B.3 Auxiliary Cost and Region Terms

A learned router assigns each transition and event a softmax distribution over $R = 6$ latent regions, denoted Q_{jr}^T and Q_{ir}^E . Each region forms normalized weighted means of its transition and

event vectors, concatenates them, and applies a small feed-forward network to obtain h_r . The denominator of each weighted mean is clamped below at 10^{-6} . Two softplus heads give $\hat{l}_r \geq 0$ and $\hat{u}_r = \hat{l}_r + \text{softplus}(w_u^\top h_r + b_u)$. These are learned endpoints, not admissible bounds. Only the sum of upper endpoints is supervised against the optimal cost:

$$\hat{c} = \sum_{r=1}^R \hat{u}_r, \quad \mathcal{L}_{\text{cost}} = H_1(\hat{c} - \delta), \quad H_1(e) = \begin{cases} \frac{1}{2}e^2, & |e| < 1, \\ |e| - \frac{1}{2}, & |e| \geq 1. \end{cases}$$

The implementation also exposes sketch and uncertainty heads; neither has a direct supervised loss or enters the current decoder.

For each place p , let U_p be the set of distinct incident transitions, regardless of arc direction. Boundary regularization averages over all unordered transition pairs within each U_p :

$$\mathcal{L}_{\text{boundary}} = \frac{\sum_{p \in P} \sum_{\{j,k\} \subseteq U_p} \left(1 - \sum_{r=1}^R Q_{jr}^T Q_{kr}^T \right)}{\sum_{p \in P} \binom{|U_p|}{2}}.$$

A pair sharing several places appears once for each such place; if there are no pairs the term is zero. It encourages incident transitions to share regions. Let the mean region load over transitions and events be $a_r = (\sum_j Q_{jr}^T + \sum_i Q_{ir}^E) / (|T| + n)$. Then

$$\mathcal{L}_{\text{balance}} = \frac{1}{R} \sum_r (a_r - 1/R)^2, \quad \mathcal{L}_{\text{entropy}} = \frac{1}{2} \left[\frac{1}{|T|} \sum_j H(Q_j^T) + \frac{1}{n} \sum_i H(Q_i^E) \right],$$

where $H(q) = -\sum_r q_r \log q_r$ (logs are clamped at 10^{-8} in code). For an empty trace, balance and entropy use only transitions. The positive entropy weight encourages concentrated assignments; balance discourages collapse into a single region.

B.4 Total Loss and Optimization Settings

The reference command uses the following complete objective:

$$\begin{aligned} \mathcal{L} = & \mathcal{L}_{\text{sync}} + \mathcal{L}_{\text{log}} + \mathcal{L}_{\text{model}} + 0.03\mathcal{L}_{\text{cost}} \\ & + 0.01\mathcal{L}_{\text{boundary}} + 0.01\mathcal{L}_{\text{balance}} + 0.001\mathcal{L}_{\text{entropy}}. \end{aligned}$$

Each component is reduced per input pair as above, then averaged over the 16 pairs in an optimizer step (the implementation processes each pair separately and accumulates gradients). AdamW uses learning rate 5×10^{-4} , weight decay 3×10^{-3} , and gradient-norm clipping at 1. Dropout is 0.30. Validation loss controls learning-rate reduction by factor 0.5 after two plateau epochs (minimum 10^{-5}) and early stopping with patience six and minimum improvement 5×10^{-4} . The seed is 13; the reference run executes 50 epochs and selects epoch 49. The settings are defined in `scripts/train_model.py`; the loss reductions and masking are defined in `lara_align/training.py`. Neither replay legality nor exact certification is a differentiable training term: both are explicit semantic checks outside the neural objective.